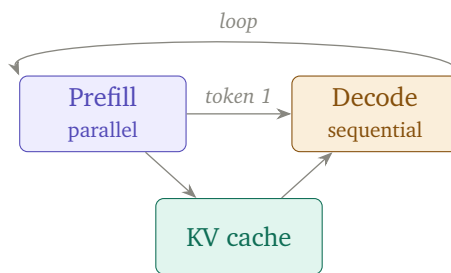

LLM Inference, Sampling, and Context Engineering

An illustrated field guide for ML systems engineers



Carmel Prosper Sagbo

Research Associate & ML Infrastructure Engineer
CyLab-Africa / Upanzi Network
Carnegie Mellon University Africa, Kigali

June 2026

For internal study and reference

Contents

How to Read This Book	vii
I Transformer Inference	1
1 The Attention Mechanism	3
1.1 The Core Idea	3
1.2 Query, Key, Value	3
1.3 The Four Steps	3
1.4 Seeing the Weights	4
1.5 Why This Connects to Everything Later	4
1.6 Multi-Head Attention	4
2 The Inference Pipeline	5
2.1 The Big Picture First	5
2.2 Two Distinct Phases	5
2.2.1 Phase 1: Prefill	5
2.2.2 Phase 2: Decode	6
2.3 Key Metrics	7
2.4 Layer Count by Model Size	7
II Hardware and Systems	9
3 The Memory Bandwidth Bottleneck	11
3.1 Why Decode Is Slow	11
3.1.1 Arithmetic Intensity	11
3.1.2 What Makes It Worse	12
4 KV Cache Memory	13
4.1 The Formula	13
4.2 Concrete Numbers	13
4.3 Weights vs. KV Cache	13
4.4 Grouped Query Attention	13
5 Mixture of Experts	15
5.1 The Core Idea	15
5.2 Active vs Total Parameters	15
5.3 Sparse Activation and Routing	16

III	Optimization	17
6	Optimization Techniques	19
6.1	KV Cache	19
6.2	Prefix Caching	19
6.3	Continuous Batching	19
6.4	Quantization	20
6.5	Flash Attention	20
6.6	Speculative Decoding	20
6.7	PagedAttention	21
6.8	Prefill–Decode Disaggregation	21
IV	Sampling and Decoding	23
7	Temperature and Sampling	25
7.1	How Temperature Reshapes the Distribution	25
7.1.1	Three Statistics to Watch	25
7.2	Task–Temperature Matrix	26
7.3	Nucleus Sampling (top_p)	26
7.4	How Temperature and top_p Interact	26
7.4.1	Common Combinations	26
7.5	Repetition Penalties	27
7.6	The JSON Extraction Case	27
V	Context Engineering	29
8	Feeding the Model Well	31
8.1	The Core Principle	31
8.2	Context Window Budgeting	31
8.3	Retrieval Ordering	31
8.4	Context Compression	32
8.5	Prompt Structure	32
8.6	Tool Output Shaping	32
8.7	RAG Quality Experiments	32
9	RAG Architecture	33
9.1	The Two Pipelines	33
9.2	Ingestion Stages	33
9.2.1	Document Ingestion	33
9.2.2	Chunking	33
9.2.3	Embedding	34
9.2.4	Vector Database	34
9.3	Understanding Embeddings	34
9.3.1	Similarity Search	34
9.3.2	Cosine Similarity	34
9.3.3	Bi-Encoders vs Cross-Encoders	35
9.4	Query Stages	35
9.4.1	Query Embedding and Retrieval	35

9.4.2	Reranking	35
9.4.3	Generation	35
9.5	Where Each Earlier Lesson Applies	36
9.6	A Closer Look at Embeddings	36
9.6.1	What an Embedding Is	36
9.6.2	Similarity Search	36
9.6.3	Bi-Encoders vs Cross-Encoders	36
9.6.4	How They Combine	37
9.6.5	Choosing an Embedding Model	38
10	Agent Context Engineering	39
10.1	Beyond Retrieval	39
10.2	Core Techniques	39
VI	Evaluation	41
11	Measuring Quality	43
11.1	Why Evaluation Is Its Own Discipline	43
11.2	Offline Metrics	43
11.3	LLM-as-Judge	43
11.4	RAG Evaluation	44
12	Reading the Benchmarks	45
12.1	Why Benchmarks Exist	45
12.2	The Major Benchmarks	45
12.3	Reading Scores Critically	45
VII	Production Architectures	47
13	Serving Architectures	49
13.1	What a Serving Engine Does	49
13.2	The Request Lifecycle	49
13.3	The Major Engines	50
13.4	Distributed Inference	50
VIII	Agents and Reasoning Models	53
14	Modern Reasoning Models	55
14.1	A Different Inference Profile	55
14.2	Core Concepts	55
14.3	Strategies for Spending Test-Time Compute	55
IX	Synthesis	57
15	The Full Stack	59
15.1	Putting It Together	59
15.2	Metrics to Track Per Experiment	59

15.3 Benchmarking Summary	59
Summary	61
X Hands-On Labs	63
16 From Reading to Doing	65
17 Lab 1 — Measure TTFT	67
18 Lab 2 — Watch KV Cache Growth	69
19 Lab 3 — Compare Temperatures	71
20 Lab 4 — Build a RAG Pipeline	73
21 Lab 5 — Evaluate the RAG System	75
Where to Go Next	77
Further Reading	79

How to Read This Book

This guide is built so that a complete beginner can follow it from cover to cover, while remaining useful as a reference for an experienced engineer. Three things make that possible.

Every concept is drawn before it is explained in depth. When a new mechanism appears — the prefill/decode split, the KV cache, the memory roofline, the RAG pipeline — there is a diagram first. The diagram gives you the shape of the idea; the prose fills in the detail.

Coloured boxes flag what kind of information you are reading:

In plain terms A blue box restates a technical idea in everyday language, with no jargon. If a section feels dense, skip to the blue box first, then come back.

A green box is a key takeaway — the single thing to remember from a section.

An amber box is a correction or caveat — a place where the simple version of the story is slightly wrong and you should know the precise version.

The book is organized in parts that build on each other: transformer inference, then the hardware that runs it, then how we make it faster, then how we control its outputs, then how we feed it good context, then how we measure it, and finally how production systems and modern reasoning models put it all together. Read in order the first time.

PART I

Transformer Inference

The Attention Mechanism

Before we trace how a model runs, we need to understand the operation at its heart. Every mention of KV cache, Flash Attention, and GQA later in this book refers back to this one mechanism. If attention stays a black box, those optimizations look like magic. They are not.

1.1 The Core Idea

A language model represents each token as a vector. Attention is how a token *looks at* every other token and pulls in the information relevant to it. The word “it” in a sentence needs to figure out what it refers to; attention is the machinery that lets “it” inspect every earlier word and decide which ones matter.

In plain terms Picture each word in a sentence asking a question and every other word holding up a label. The asking word compares its question against all the labels, pays most attention to the labels that match, and gathers information mostly from those. “Attention” is just this matching-and-gathering, done with numbers.

1.2 Query, Key, Value

Each token produces three vectors by multiplying its representation through three learned weight matrices (W_Q , W_K , W_V):

- **Query (Q)** — what this token is looking for.
- **Key (K)** — what this token offers, used for matching against other tokens’ queries.
- **Value (V)** — the actual information this token passes on if attended to.

The metaphor that makes this stick: a Query is a search box, Keys are the titles of documents, and Values are the documents’ contents. You match your search against titles, then read the contents of the best matches.

1.3 The Four Steps

For a single query token attending to all tokens:



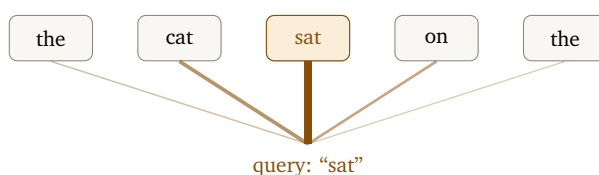
1. **Score.** Take the dot product of the query with every key. A large dot product means “this key is relevant to my query.”
2. **Scale.** Divide by $\sqrt{d_{\text{head}}}$ to keep the numbers stable (large dimensions otherwise produce huge dot products).
3. **Softmax.** Convert the raw scores into a probability distribution that sums to 1 — the *attention weights*. High-scoring tokens get most of the weight.
4. **Weighted sum.** Multiply each token’s Value by its attention weight and add them up. The result is the query token’s new representation, now infused with information from the tokens it attended to.

Written compactly, this is the famous scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d_{\text{head}}}}\right) V$$

1.4 Seeing the Weights

The picture below shows one query token (“sat”) attending to every token in “the cat sat on the”. Line thickness is the attention weight — how much “sat” draws from each token.



The thicker the line, the more “sat” incorporates that token’s Value into its updated representation. This is repeated for every token, in every attention head, in every layer.

1.5 Why This Connects to Everything Later

The Key and Value vectors are exactly what the **KV cache** stores. Once computed for a token, they never change — so we cache them rather than recompute them every decode step. **GQA** shrinks the cache by sharing K and V across query heads. **Flash Attention** computes that softmax-and-weighted-sum without ever writing the full score matrix to memory. Every optimization in Part III is an optimization of *this one operation*.

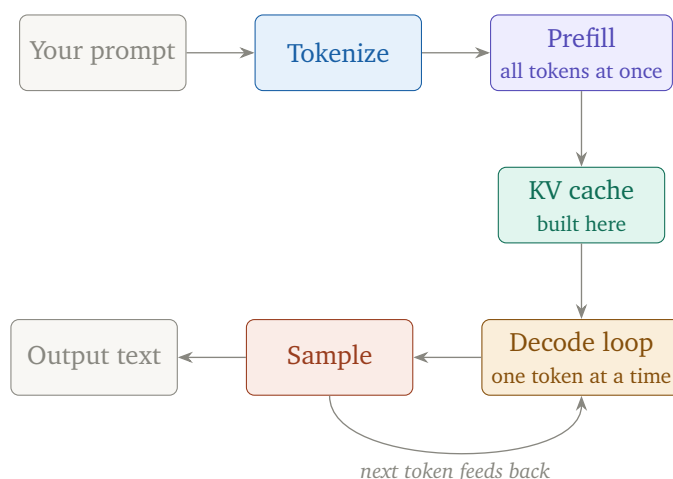
1.6 Multi-Head Attention

Rather than one attention operation, models run several in parallel — each a “head” with its own W_Q , W_K , W_V . One head might track grammatical subject, another long-range references, another local word order. Their outputs are concatenated and projected back to the model dimension. This is why model specs list a head count (for example, 32 query heads in Llama 3.1 8B).

The Inference Pipeline

2.1 The Big Picture First

Before any detail, here is the whole journey a request takes, from the moment you hit send to the moment the last word appears.



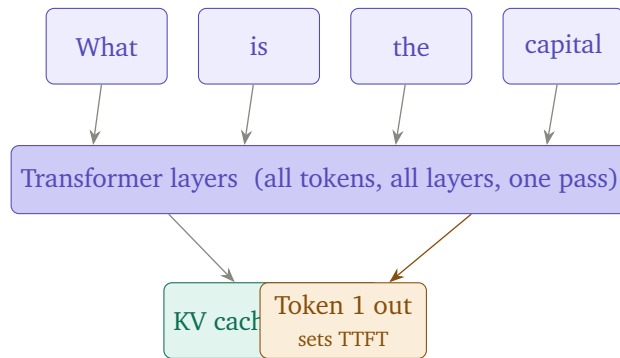
In plain terms Think of the model as a very well-read person answering a question. First they read your entire question in one glance (**prefill**). Then they write the answer one word at a time (**decode**), where each new word depends on everything written so far. The reading is fast and happens all at once. The writing is slow and strictly one-word-after-another.

2.2 Two Distinct Phases

Every LLM request passes through two computationally different phases. Their asymmetry is the most important mental model in this entire book — it explains almost every optimization that follows.

2.2.1 Phase 1: Prefill

During prefill, every input token is processed *simultaneously* in one large forward pass through all transformer layers. The prompt is not read word by word; all tokens enter together and flow through the layers in parallel.



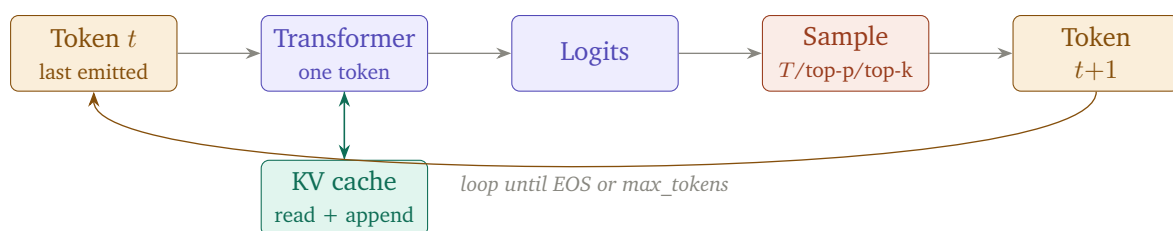
What happens to each token during prefill:

1. **Embedding lookup.** Each token ID maps to a dense vector (a row of the embedding matrix). For a 7B model this vector is typically 4096 dimensions. The batch of tokens forms a matrix of shape $[\text{seq_len} \times d_{\text{model}}]$.
2. **Positional encoding.** A positional signal (RoPE in modern models) tells the model where each token sits.
3. **Transformer layers ($\times N$).** Each layer has two sub-layers:
 - *Attention:* each token computes Query, Key, Value vectors; every Query attends to every Key; the result is a weighted sum of Values. The full $[\text{seq_len} \times \text{seq_len}]$ attention matrix is computed at once.
 - *Feedforward (FFN):* a 2-layer MLP applied independently to each token, expanding the hidden dimension $\sim 4\times$ then contracting.
4. **KV cache write.** The Key and Value tensors at each layer are saved. This is the only time they are ever computed.
5. **Logits.** The last token's hidden state is projected through the LM head ($d_{\text{model}} \times \text{vocab}$). The first token is sampled. Its arrival is TTFT.

Prefill is compute-bound. The large matrix multiplications keep the GPU's tensor cores saturated. Cost scales as $O(\text{seq_len}^2 \times d_{\text{model}})$ for attention.

2.2.2 Phase 2: Decode

After token 1, the model enters the decode loop. This phase is strictly serial: one forward pass per output token, with no way to parallelize across tokens within a single request.



Decode is memory-bandwidth-bound. Each step does little arithmetic but must read the entire KV cache from VRAM. The GPU spends most of each step waiting for data, not computing.

In plain terms Prefill is like photographing a whole page at once — fast, all-at-once. Decode is like writing the reply by hand: each letter is quick, but you cannot write letter five before letter four, and before each letter you must re-read everything you have written so far. That re-reading is the slow part.

2.3 Key Metrics

Metric	Description
TTFT	Time To First Token. Dominated by prefill once the request reaches the model.
Tokens/sec	Decode generation speed. Limited by memory bandwidth.
Throughput	Requests per second across all users. Improved by batching.
End-to-end latency	TTFT + (output tokens $\times$ time per decode step).
GPU memory	Model weights (fixed) + KV cache (grows per request and per token).
Cost per token	Combined GPU time, memory, and operational overhead.

2.4 Layer Count by Model Size

Model	Layers	d_{model}	Parameters
Llama 3.2 1B	16	2048	1B
Llama 3.2 3B	28	3072	3B
Llama 3.1 8B	32	4096	8B
Mistral 7B	32	4096	7B
Llama 3.1 70B	80	8192	70B
Llama 3.1 405B	126	16384	405B
GPT-3	96	12288	175B

Layer count is a hard multiplier on decode latency: layer $n + 1$ cannot start until layer n finishes. A 70B model (80 layers) takes roughly $2.5\times$ longer per decode step than an 8B model (32 layers), all else equal. This is why large models use *tensor parallelism* (splitting each layer across GPUs) rather than pipelining across layers for a single token.

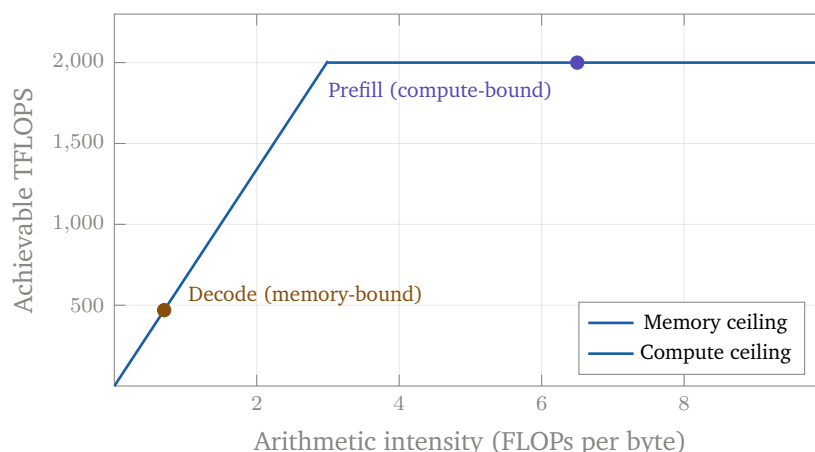
PART II

Hardware and Systems

The Memory Bandwidth Bottleneck

3.1 Why Decode Is Slow

A modern GPU is a compute monster with a comparatively narrow memory pipe. An H100 delivers roughly 2,000 TFLOPS of FP16 compute but only about 3.35 TB/s of memory bandwidth. In decode, the narrow pipe is the binding constraint.



The diagonal line is the *memory ceiling*: performance you can reach when limited by bandwidth. The flat line is the *compute ceiling*. Prefill, with many tokens per pass, has high arithmetic intensity and sits up against the compute ceiling. Decode, with one token per pass, has low arithmetic intensity and is pinned far down the memory diagonal — the GPU’s compute cores sit mostly idle.

In plain terms Imagine a chef (the GPU cores) who can chop incredibly fast, but the ingredients arrive on a slow conveyor belt (memory bandwidth). When cooking one dish at a time (decode), the chef spends most of the time waiting for the belt. When prepping a huge banquet at once (prefill), the belt is always full and the chef is the limit. Decode keeps the fast chef waiting.

3.1.1 Arithmetic Intensity

The deciding ratio is **arithmetic intensity**: FLOPs performed per byte read from memory.

- **Prefill**: high intensity. Many tokens $\Rightarrow$ large matmuls $\Rightarrow$ cores saturated.
- **Decode**: very low intensity. One token $\Rightarrow$ thin matmuls $\Rightarrow$ cores wait on HBM.

3.1.2 What Makes It Worse

The KV cache grows by one row (per layer, per head) every decode step. At step t , attention must read tensors of shape $[n_{\text{layers}} \times n_{\text{heads}} \times t \times d_{\text{head}}]$ from HBM. As t grows, the bytes moved per step grow linearly — long generations get progressively more bandwidth-starved.

KV Cache Memory

4.1 The Formula

For a single request at FP16:

$$\text{KV cache bytes} = 2 \times n_{\text{layers}} \times n_{\text{kv_heads}} \times d_{\text{head}} \times \text{seq_len} \times 2$$

The leading 2 is for K and V; the trailing 2 is bytes per FP16 element.

This formula is for a **single sequence**. Production servers handle many requests at once, so the real figure carries a `batch_size` multiplier: multiply the whole expression by the number of concurrent sequences. A server running 32 concurrent requests uses $32\times$ the KV memory of the tables below.

4.2 Concrete Numbers

Model	Layers	KV/token	2k	8k	32k
Llama 3.1 8B	32	0.5 MB	1 GB	4 GB	16 GB
Llama 3.1 70B	80	2.5 MB	5 GB	20 GB	80 GB
Llama 3.1 405B	126	8 MB	16 GB	64 GB	256 GB

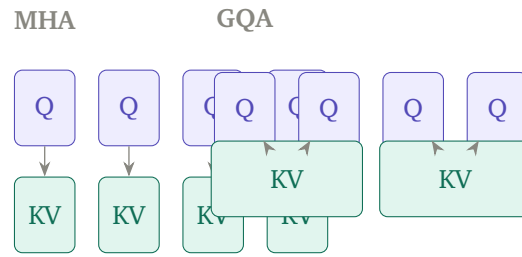
The 70B model at 32k context uses 80 GB for one request's KV cache — the whole of an A100 80GB, before weights.

4.3 Weights vs. KV Cache

Model	Weights (FP16)	Weights (INT4)
8B	~16 GB	~4 GB
70B	~140 GB	~35 GB
405B	~810 GB	~200 GB

4.4 Grouped Query Attention

Modern models shrink the KV cache by sharing K and V across groups of query heads.



MHA : $n_{\text{heads}} \times d_{\text{head}} \times n_{\text{layers}}$ (e.g. 32 KV heads)

GQA : $n_{\text{kv_heads}} \times d_{\text{head}} \times n_{\text{layers}}$ (e.g. 8 KV heads, $4 \times$ smaller)

MQA : $1 \times d_{\text{head}} \times n_{\text{layers}}$ (extreme, $32 \times$ smaller)

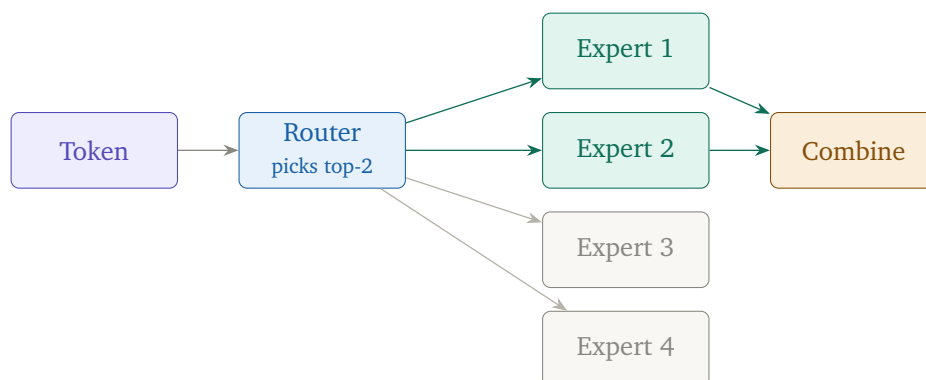
Llama 3.1 8B uses 8 KV heads against 32 query heads — a $4 \times$ KV reduction with negligible quality loss. This is one of the highest-leverage inference decisions in modern architectures.

Mixture of Experts

So far we have treated a model’s parameter count as if every parameter runs on every token. For *dense* models that is true. But many of the most capable recent models are *sparse*: they have enormous total parameter counts yet only activate a small fraction per token. This is why a model can be “671B parameters” and still be cheaper to run than a 70B dense model. The mechanism is Mixture of Experts (MoE).

5.1 The Core Idea

In a dense transformer, the feedforward (FFN) sub-layer is a single large network every token passes through. In an MoE layer, that single FFN is replaced by many smaller FFNs — the *experts* — plus a small *router* that sends each token to just a few of them.



Here the router activates 2 of 4 experts. Real systems are larger — DeepSeek-V3 uses 256 routed experts and activates 8 per token — but the principle is identical: each token only pays for the experts it actually visits.

In plain terms A dense model is one giant generalist who answers every question personally. An MoE model is a large office of specialists with a receptionist (the router). Each question is sent to the two or three specialists best suited to it. The office is huge, but any single question only ever occupies a few people — so it is fast despite the size.

5.2 Active vs Total Parameters

This is the number that confuses newcomers. An MoE model has two parameter counts:

- **Total parameters:** every expert in the whole model. Determines memory needed to *store* the model.
- **Active parameters:** only the experts (plus shared layers) used per token. Determines the *compute* per token.

Model	Total	Active	Behaves like
Mixtral 8x7B	47B	13B	a 13B model in speed
DeepSeek-V3	671B	37B	a 37B model in speed

DeepSeek-V3 has the knowledge capacity of a 671B model but the per-token compute cost of a 37B model. That is the entire appeal of MoE.

MoE saves *compute*, not *memory*. All experts must be resident in VRAM because the router might pick any of them for the next token. DeepSeek-V3's 671B parameters still occupy 671B-worth of memory (~ 350 GB even quantized) — you simply do less arithmetic per token. The bottleneck shifts from FLOPs toward memory capacity and the bandwidth to stream the chosen experts.

5.3 Sparse Activation and Routing

The router is a small learned layer that scores each token against every expert and selects the top- k (typically $k = 2$). Two practical concerns dominate MoE engineering:

- **Load balancing:** if the router sends most tokens to a few favourite experts, the others sit idle and the benefit evaporates. Training uses an auxiliary load-balancing loss to spread tokens evenly.
- **Expert parallelism:** experts are distributed across GPUs, so routing a token to an expert may mean sending it across the network. This adds a communication pattern (all-to-all) not present in dense models, and is a major focus of MoE serving systems.

MoE is the clearest case of the book's recurring theme: the binding constraint is rarely raw compute. MoE trades abundant memory capacity for scarce compute, then spends serving effort managing the routing and communication that trade creates.

PART III

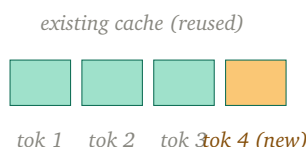
Optimization

Optimization Techniques

Every technique here attacks one of two enemies: the memory-bandwidth cost of decode, or the strictly-sequential nature of decode.

6.1 KV Cache

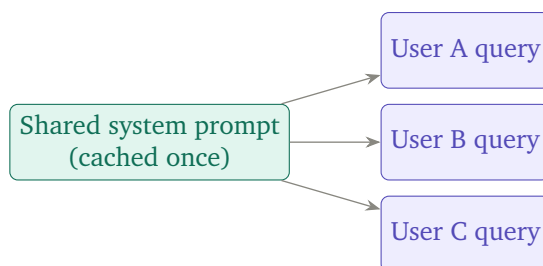
Without caching, each decode step would recompute Key and Value matrices for every previous token — $O(t)$ wasted work per step. The cache stores them after prefill and appends one row per step.



Memory cost: $2 \times n_{\text{layers}} \times n_{\text{kv_heads}} \times d_{\text{head}} \times t \times \text{bytes}$. At long sequences this dominates VRAM.

6.2 Prefix Caching

If many requests share a byte-identical prefix (a system prompt, a shared document), its KV cache is computed once and reused across all of them.

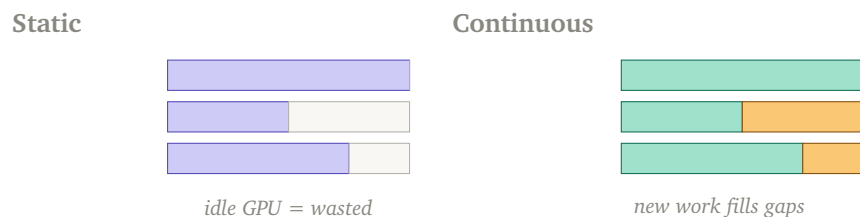


For a 2,000-token system prompt shared by 10,000 daily users, prefix caching eliminates essentially all prefill cost after the first request — the most compute-intensive part of every request becomes free.

6.3 Continuous Batching

Static batching holds the GPU until every sequence in the batch finishes, leaving cores idle when some finish early. Continuous batching evicts finished sequences and admits new ones at *every*

decode step.



Result: near-100% GPU utilization and far higher throughput, at the cost of slightly higher per-request latency variance.

6.4 Quantization

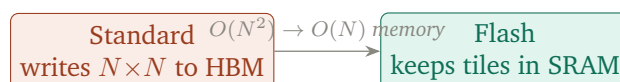
Reducing weight precision cuts both memory footprint and bandwidth per step.

Precision	Memory savings	Quality
FP16	baseline	full
INT8 (LLM.int8, GPTQ)	$\sim 2\times$	minimal loss on most tasks
INT4 (AWQ, GGUF Q4)	$\sim 4\times$	watch for reasoning degradation

KV-cache quantization (separate from weight quantization) directly multiplies how many concurrent requests fit in VRAM.

6.5 Flash Attention

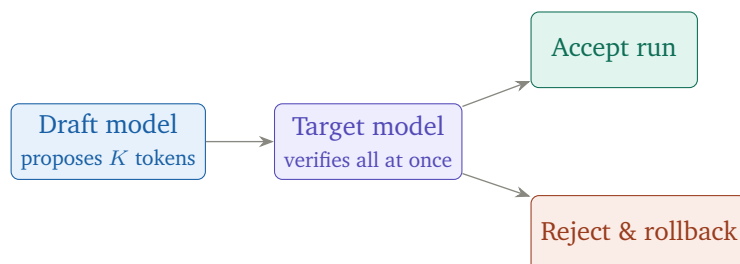
Standard attention writes the full $N \times N$ attention matrix to HBM. Flash Attention fuses softmax and matmul into one CUDA kernel that keeps intermediates in on-chip SRAM, never materializing the full matrix.



Result: memory footprint $O(N)$ instead of $O(N^2)$, and 2–4 $\times$ faster attention on A100/H100. Now the default in most frameworks.

6.6 Speculative Decoding

A small draft model proposes K tokens; the large target model verifies all K in one forward pass (like a mini-prefill). Accepted tokens are kept; the first rejection rolls back.



Net 2–3 $\times$ speedup when the draft model is often right (common phrases, code). Costs draft-model VRAM plus verification logic.

6.7 PagedAttention

Before PagedAttention (vLLM), servers pre-allocated the maximum context length for every request, wasting VRAM on sequences that ended early. PagedAttention treats KV memory like OS virtual-memory pages: fixed blocks allocated only as needed. This eliminates fragmentation and sharply raises concurrent capacity.

6.8 Prefill–Decode Disaggregation

When prefill and decode share a batch, a long prefill stalls in-flight decode steps — *prefill–decode interference*, seen as latency spikes. The modern fix runs prefill on dedicated instances and decode on separate instances, routing between them so each phase is tuned independently.

PART IV

Sampling and Decoding

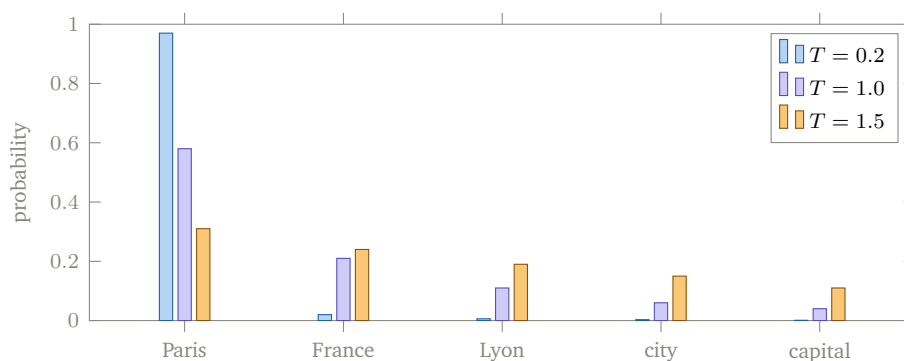
Temperature and Sampling

7.1 How Temperature Reshapes the Distribution

Temperature T divides every logit before the softmax:

$$p_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

The picture below shows the same five-token distribution at three temperatures. Low T makes one token dominate; high T flattens everything toward uniform.



In plain terms Temperature is a confidence dial. At low temperature the model is decisive — it almost always picks its single favourite word. At high temperature it becomes adventurous, willing to pick less likely words. Low T for facts and code; high T for brainstorming and fiction.

7.1.1 Three Statistics to Watch

- **Top token probability:** how dominated the distribution is. $T=0.2 \Rightarrow 85\text{--}95\%$; $T=1.5 \Rightarrow \sim 30\%$.
- **Entropy** $H = -\sum_i p_i \log_2 p_i$: $0 = \text{deterministic}$; $\text{max} = \log_2 V$.
- **Effective vocabulary** 2^H : “how many tokens is the model really considering?” $T=0.2 \Rightarrow 1\text{--}2$; $T=1.5 \Rightarrow 8\text{--}12$.

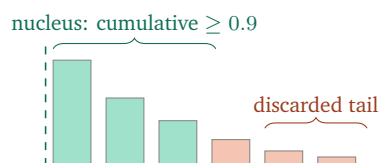
7.2 Task–Temperature Matrix

Task	T	Rationale
JSON extraction	0.0	determinism required; variance is failure
Classification	0.0–0.2	answer is singular
Coding	0.1–0.3	correctness over variety
Analysis / summary	0.2–0.4	nuance, still stable
Conversational	0.5–0.8	natural variation without chaos
Brainstorming	0.8–1.2	diversity valued
Creative writing	0.8–1.5	novelty is the point

Most production systems live in $T \in [0.0, 0.3]$: consistency beats creativity for structured work.

7.3 Nucleus Sampling (top_p)

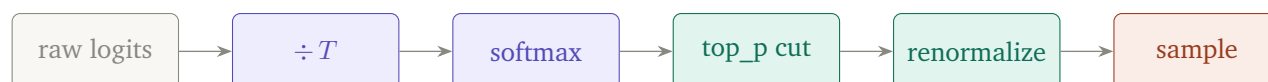
Sort tokens by probability descending; keep the smallest set whose cumulative mass $\geq p$; renormalize; sample.



Unlike top_k (a fixed count), top_p adapts: when the model is confident the nucleus is small; when uncertain it widens.

7.4 How Temperature and top_p Interact

They compose in sequence — top_p only ever sees what temperature produced.



What top_p sees depends entirely on temperature. At $T=0.2$ the distribution is already peaked, so top_p=0.9 captures 1–2 tokens and does almost nothing. At $T=1.5$ the distribution is flat, so top_p=0.9 captures 12–15 tokens and acts as a safety net, cutting the low-probability tail that temperature inflated.

7.4.1 Common Combinations

T	top_p	Use case
0.0	any	greedy; top_p irrelevant
0.2	1.0	near-deterministic; top_p irrelevant
1.0	0.95	standard creative; trims bottom 5% tail
1.2	0.9	elevated creativity with a tail cutoff

Rule: tune temperature to set creativity; use top_p=0.95 as a fixed guard rail. Temperature is the creativity dial; top_p is the filter.

7.5 Repetition Penalties

- **Presence penalty:** lowers the logit of any token seen at least once. Encourages new topics.
- **Frequency penalty:** lowers the logit in proportion to how often a token appeared. Kills loops and filler.

Start at presence=0.1, frequency=0.1; raise for long-form generation.

7.6 The JSON Extraction Case

For structured JSON extraction: $T = 0$, full stop. The value is in the source or it is not — variance is failure. Beyond temperature:

- Give the exact schema with field names, types, and a filled example.
- Tell the model to return `null` for absent fields rather than hallucinate.
- Use grammar-constrained decoding (vLLM, llama.cpp) where available — this makes malformed JSON *impossible*, not merely unlikely.

PART V

Context Engineering

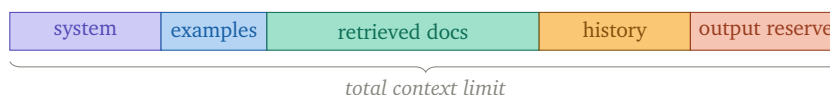
Feeding the Model Well

8.1 The Core Principle

Better context > bigger model for most production applications. The quality of what you put *into* the context window often matters more than the parameter count of the model reading it.

8.2 Context Window Budgeting

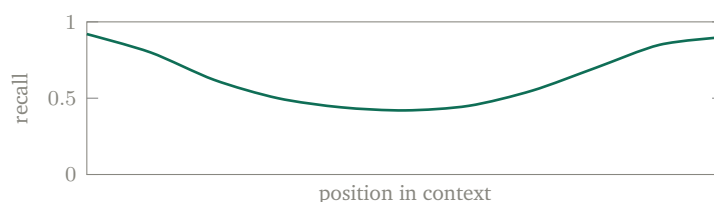
The window is shared among several competing demands:



Reserve 20–30% for output. Prune the cheapest inputs first (summarize history, truncate documents) before sacrificing retrieval quality.

8.3 Retrieval Ordering

LLMs suffer a “lost in the middle” effect: attention to content sags in the middle of a long context. Put the most relevant chunks **first**. Never sort by row ID or timestamp; always sort by relevance score descending. This alone can lift answer quality with no model change.



8.4 Context Compression

Strategy	Method	Best for
Extractive	keep sentences above a relevance threshold	long docs, sparse relevance
Abstractive	summarize docs before inserting	dense narrative docs
Selective	cheap model filters chunks before the expensive one	high-throughput pipelines

Target a 10:1 compression ratio while keeping >90% of answer-relevant content.

8.5 Prompt Structure

```
## Role - who the model is
## Task - what to do (an action)
## Constraints - what to avoid, scope limits
## Evidence - retrieved context inserted here
## Output - schema or concrete example
```

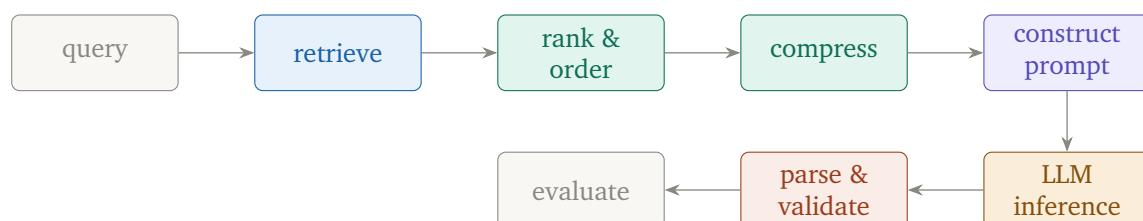
Consistent structure across calls produces consistent outputs — vital for extraction and classification.

8.6 Tool Output Shaping

```
# Instead of dumping a 40-field API response:
{"relevant_facts": [{"date": "2026-06-18", "temp": "28C", "sunny": true}]}
```

Filter tool outputs to the fields the model needs. This cuts token cost and removes distracting noise.

8.7 RAG Quality Experiments



Version A: dump all docs, no structure. Baseline.

Version B: add Role/Task/Constraints/Evidence/Format. Measure gain over A.

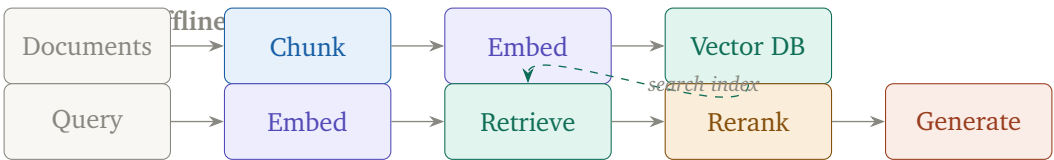
Version C: add relevance ordering + compression. Measure token reduction and quality gain over B.

Run the same 10 test questions across all three. The A-to-C delta is often larger than upgrading from a 7B to a 70B model.

RAG Architecture

The previous chapter covered how to make a RAG system produce *good* answers. This chapter covers how a RAG system is *built* — the full path from raw documents to a grounded answer. The two halves of the pipeline run at different times: ingestion happens offline, ahead of any query; retrieval and generation happen live, per request.

9.1 The Two Pipelines



In plain terms Ingestion is like building a library: you take in books, cut them into findable sections, label each section by meaning, and shelve them. A query is like a visitor’s question: you translate the question into the same labelling scheme, fetch the closest sections, pick the best few, and hand them to the model to write the answer.

9.2 Ingestion Stages

9.2.1 Document Ingestion

Raw sources (PDFs, HTML, databases, transcripts) are parsed into clean text. This stage is unglamorous but decisive: bad parsing (broken tables, lost headings, OCR errors) poisons everything downstream. Preserve structure — section titles, lists, tables — because it carries meaning.

9.2.2 Chunking

Documents are split into retrievable pieces. The chunk size is a genuine tradeoff:

Strategy	Behaviour
Fixed-size	split every N tokens. Simple, but cuts across ideas.
Sentence / paragraph	split on natural boundaries. Preserves meaning, variable size.
Recursive	split on structure (headings → paragraphs → sentences) until chunks fit.
Overlapping windows	adjacent chunks share tokens so context is not lost at boundaries.

Small chunks retrieve precisely but lose context; large chunks carry context but dilute relevance and waste tokens. Overlap (typically 10–20%) hedges against splitting a key fact across a boundary.

9.2.3 Embedding

Each chunk is passed through an embedding model that maps text to a dense vector capturing its meaning. Chunks with similar meaning land near each other in vector space. The choice of embedding model matters as much as the generation model — a weak embedder retrieves the wrong chunks no matter how good the LLM is.

9.2.4 Vector Database

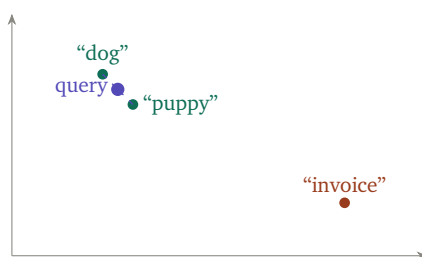
Embeddings are stored in a vector database (Qdrant, Pinecone, Weaviate, pgvector, FAISS) that supports fast approximate nearest-neighbour search. At query time it returns the chunks whose vectors are closest to the query vector, typically via an index like HNSW that trades a little accuracy for large speed gains.

9.3 Understanding Embeddings

Retrieval quality often depends more on the embedding model than on the generator. A weak embedder retrieves the wrong chunks, and no LLM can answer well from the wrong evidence. This section unpacks how embeddings actually drive retrieval.

9.3.1 Similarity Search

An embedding maps text to a point in a high-dimensional space (commonly 384 to 3072 dimensions) such that semantically similar texts land near each other. Retrieval is then a geometry problem: find the chunk vectors nearest to the query vector.



The query lands near “dog” and “puppy” (close in meaning) and far from “invoice”. The two near neighbours are retrieved.

9.3.2 Cosine Similarity

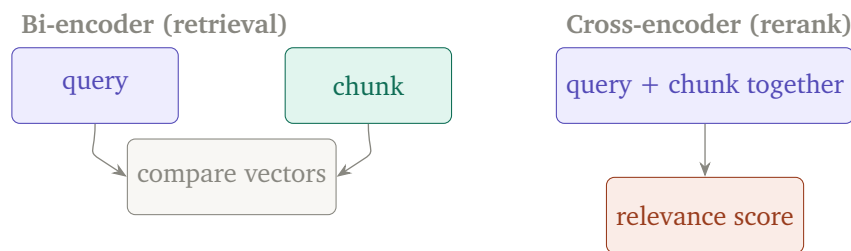
The standard distance metric is cosine similarity: the cosine of the angle between two vectors. It measures *direction*, ignoring magnitude, which suits text well because what matters is meaning, not vector length.

$$\cos(\theta) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|}$$

A value near 1 means very similar; near 0 means unrelated. Many vector DBs normalize vectors to unit length so that cosine similarity reduces to a simple dot product, which is faster to compute at scale.

9.3.3 Bi-Encoders vs Cross-Encoders

This distinction is the technical heart of the retrieve-then-rerank pattern.



- **Bi-encoder:** encodes query and chunk *separately* into vectors, then compares. Because chunk vectors are precomputed at ingestion, search is extremely fast — this is what powers retrieval over millions of chunks.
- **Cross-encoder:** feeds query and chunk *together* through the model and outputs a single relevance score. Far more accurate because it can attend across both texts, but far slower — it cannot precompute, so every (query, chunk) pair is a fresh forward pass.

This is exactly why production RAG retrieves with a bi-encoder (fast, wide net, high recall) and reranks with a cross-encoder (slow, precise, high precision). The two encoder types are the implementation of the precision/recall division of labour described earlier.

9.4 Query Stages

9.4.1 Query Embedding and Retrieval

The user query is embedded with the *same* model used for chunks, then the vector DB returns the top- k nearest chunks. A common refinement is **hybrid retrieval**: combine dense vector search with classic keyword search (BM25) to catch both semantic matches and exact-term matches.

9.4.2 Reranking

The top- k retrieved chunks are often reordered by a cross-encoder reranker that scores each (query, chunk) pair directly. Retrieval is fast but approximate; reranking is slower but precise. The pattern is: retrieve 50 candidates cheaply, rerank to the best 5 accurately.

Retrieval and reranking are a precision/recall division of labour. Retrieval maximizes *recall* — cast a wide net so the right chunk is somewhere in the candidates. Reranking maximizes *precision* — pick the few chunks that truly answer the query. This connects directly to the context-precision and context-recall metrics in the evaluation chapter.

9.4.3 Generation

The reranked chunks become the Evidence section of the prompt (recall the Role/Task/Constraints/Evidence/Format structure), ordered most-relevant-first to fight the lost-in-the-middle effect. The LLM generates a grounded answer, ideally citing which chunks it used so faithfulness can be checked.

9.5 Where Each Earlier Lesson Applies

Stage	Earlier concept it uses
Chunking	context window budgeting — chunk size feeds token budget
Retrieval	context recall metric
Reranking	context precision metric, retrieval ordering
Generation	prompt structure, $T=0-0.3$, faithfulness evaluation
Serving	the live pipeline runs through the serving engine of Part VII

9.6 A Closer Look at Embeddings

Retrieval quality often depends more on the embedding model than on the generator. A perfect LLM cannot answer from chunks the retriever never found. This section unpacks the machinery the ingestion pipeline glossed over.

9.6.1 What an Embedding Is

An embedding model maps a piece of text to a fixed-length vector (typically 384 to 3072 dimensions) such that texts with similar meaning land near each other in that space. “How do I reset my password?” and “I forgot my login credentials” should produce nearby vectors despite sharing few words.

9.6.2 Similarity Search

Retrieval compares the query vector against stored chunk vectors using a similarity measure. The dominant choice is **cosine similarity**, which measures the angle between two vectors and ignores their magnitude:

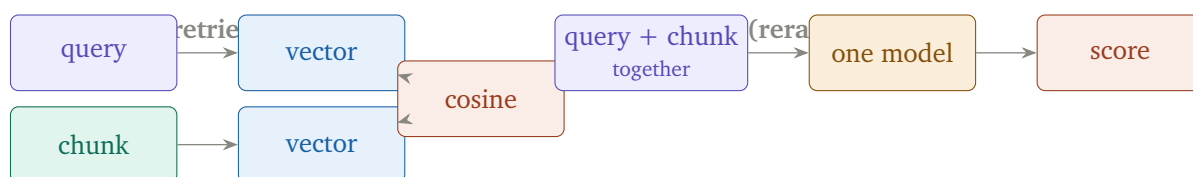
$$\cos(\theta) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|}$$

A value near 1 means very similar direction (similar meaning); near 0 means unrelated. Ignoring magnitude matters because document length should not dominate relevance.

In plain terms Think of every text as an arrow in a huge space. Cosine similarity asks “do these two arrows point the same way?” rather than “are they the same length?” Two passages about the same topic point the same way even if one is a sentence and the other a page.

9.6.3 Bi-Encoders vs Cross-Encoders

This distinction is exactly the retrieve-then-rerank split from the previous section, viewed from the model side:



- **Bi-encoder:** encodes query and chunk *separately* into vectors, then compares. Chunk vectors are precomputed at ingestion, so search is fast — this is what makes vector-DB retrieval scale to millions of documents.

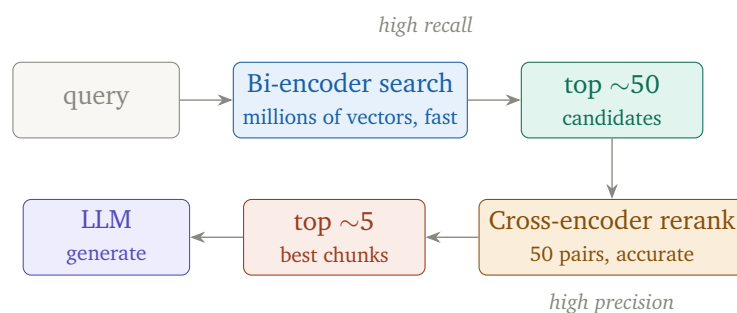
- **Cross-encoder:** feeds query and chunk *together* through one model that directly outputs a relevance score. Far more accurate because the two texts can interact through attention, but it cannot precompute anything — every (query, chunk) pair is a fresh forward pass.

The names refer to *how text is encoded*, not how many models are used. Both typically use a single model; the difference is whether the query and chunk are processed *separately* (bi) or *jointly* (cross). A bi-encoder may occasionally use two separately-trained towers (a “dual-encoder”), but the defining feature is always separate encoding, not the model count.

The pattern falls out naturally: use the fast bi-encoder to retrieve a wide candidate set from millions of chunks (high recall), then the accurate cross-encoder to rerank a few dozen candidates to the final handful (high precision). You cannot afford a cross-encoder over millions of chunks, and you cannot trust a bi-encoder’s ranking alone.

9.6.4 How They Combine

The two are not competing choices — they run together, in sequence, and that combination is the standard production setup for high-quality retrieval. Each does the job it is suited to.



The bi-encoder casts a wide net cheaply: it scans millions of precomputed vectors in milliseconds, but its ranking is approximate because query and chunk were never encoded together. So you deliberately ask it for *more* candidates than you need — trading a little precision for recall, just to ensure the right chunk lands somewhere in the top 50. The cross-encoder then reranks only those 50, where it can afford to be slow and accurate: fifty fresh forward passes is cheap, whereas fifty million would be impossible. You keep the top handful and pass them to the LLM.

The order cannot be swapped. The bi-encoder must go first because it is the only stage that scales to a large corpus; the cross-encoder must go second because it only works on a small candidate set. Each covers the other’s weakness — fast search *and* precise final ranking.

A strong production stack often extends the first stage to **hybrid retrieval**: combine the bi-encoder’s dense vector search with classic keyword search (BM25) to catch both semantic and exact-term matches, then rerank with the cross-encoder. The full pattern is: hybrid retrieve → cross-encoder rerank → generate. The reranker adds a model to host and a little latency, so it earns its place when answer quality matters enough to justify the hop — legal, medical, technical support, anywhere a slightly wrong chunk yields a confidently wrong answer.

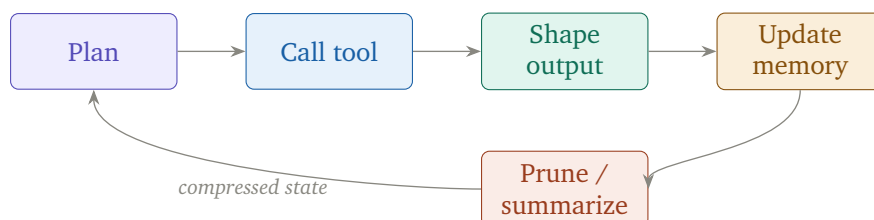
9.6.5 Choosing an Embedding Model

Selection criteria that matter in practice: the dimension (higher captures more nuance but costs more storage and search time), the maximum input length (must exceed your chunk size), the domain (a model trained on general web text may underperform on legal or medical jargon), and whether it is multilingual if your corpus is. Public leaderboards such as MTEB compare embedding models across retrieval, clustering, and classification tasks.

Agent Context Engineering

10.1 Beyond Retrieval

Classic context engineering is mostly RAG. Production agents extend it to managing *evolving state* across many turns and tool calls. “Context engineering” at frontier labs now means this broader discipline.



10.2 Core Techniques

- **Tool calling:** the model emits a structured call; the result is fed back as context. Schema discipline matters as much as in extraction.
- **State compression:** as a conversation grows, raw history overflows the window. Summarize old turns into a compact running state.
- **Memory summarization:** distill durable facts (user preferences, decisions) into a small persistent memory separate from turn-by-turn history.
- **Conversation state:** track what has been decided, what is pending, and what tools have returned — explicitly, not implicitly in raw transcript.
- **Context pruning:** drop tool outputs and turns no longer relevant to the current goal before each new model call.
- **Tool output shaping:** same discipline as RAG — return only the fields the next step needs.

In plain terms A chatbot that just remembers everything will eventually overflow and slow down. A good agent is like a tidy notetaker: it keeps a short running summary of what matters, files away durable facts, and throws out scratch work once it is done. Managing that notebook well is “agent context engineering”.

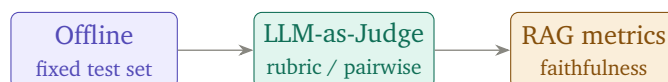
PART VI

Evaluation

Measuring Quality

11.1 Why Evaluation Is Its Own Discipline

You cannot optimize what you cannot measure. Once a system is deployed, “it looks good” stops being acceptable — you need numbers that move when quality moves.



11.2 Offline Metrics

Metric	What it measures
Exact Match	output equals reference string exactly. Brittle but unambiguous.
F1	token overlap between output and reference (precision + recall).
ROUGE	recall-oriented n-gram overlap; standard for summarization.
BLEU	precision-oriented n-gram overlap; standard for translation.
BERTScore	semantic similarity via embeddings, not surface overlap.

11.3 LLM-as-Judge

When references do not exist or quality is subjective, use a strong model to score outputs:

- **Pairwise ranking:** judge picks the better of two responses. Robust to scale bias.
- **Rubric scoring:** judge rates against explicit criteria (accuracy, completeness, tone).

LLM judges have biases — position bias (favouring the first option), length bias (favouring longer answers), and self-preference. Randomize order, control for length, and validate the judge against human labels on a sample.

11.4 RAG Evaluation

Metric	Question it answers
Context precision	of the retrieved chunks, how many were relevant?
Context recall	of the relevant chunks that exist, how many were retrieved?
Answer faithfulness	is the answer grounded in the retrieved context (no hallucination)?
Answer relevance	does the answer actually address the question?

Tooling: Ragas (RAG metrics), DeepEval (unit-test-style eval), LangSmith (tracing and eval harness).

Reading the Benchmarks

Model announcements quote a wall of benchmark scores. This short chapter is a decoder ring: what each major benchmark actually measures, so you can read a release and know whether it is relevant to your use case.

12.1 Why Benchmarks Exist

A benchmark is a fixed test set with a scoring rule, used to compare models on equal footing. No single benchmark captures “how good” a model is; each probes a specific capability. The skill is matching the benchmark to the capability you care about.

12.2 The Major Benchmarks

Benchmark	Tests	What a high score means
MMLU	broad knowledge	multiple-choice across 57 subjects; general factual and reasoning breadth
GPQA	hard reasoning	graduate-level science questions designed to resist web lookup
HumanEval	code generation	writes functions that pass hidden unit tests; basic coding ability
SWE-bench	real software	resolves actual GitHub issues in real repos; practical engineering competence
Aider Polyglot	code editing	multi-language editing in a real workflow; agentic coding skill

In plain terms MMLU is a broad trivia-and-reasoning exam. GPQA is a hard science final designed so you cannot just search the answer. HumanEval is “write this function.” SWE-bench is “fix this real bug in this real codebase,” which is far harder and far more telling for engineering work.

12.3 Reading Scores Critically

Benchmarks are noisy signals, not ground truth. Watch for: **contamination** (test questions leaked into training data inflate scores), **saturation** (once everyone scores 90%+, the benchmark stops discriminating), **prompt sensitivity** (the same model can swing several points with different prompting), and **misaligned tasks** (a high MMLU score says little about coding or agentic ability).

Always ask what was measured and whether it matches your workload.

The trajectory worth noting: easier benchmarks like MMLU and HumanEval have largely saturated, so the field has moved toward harder, contamination-resistant, real-world tests like GPQA and SWE-bench. When comparing two models, weight the benchmark that most resembles your actual task, and treat a single headline number with healthy skepticism.

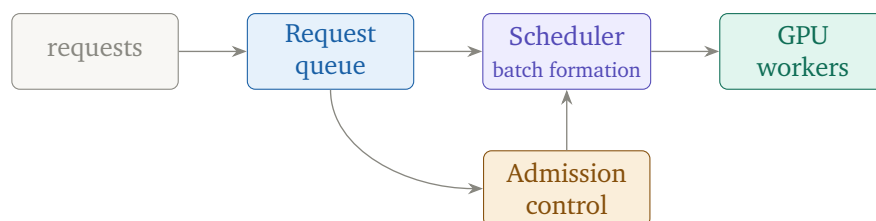
PART VII

Production Architectures

Serving Architectures

13.1 What a Serving Engine Does

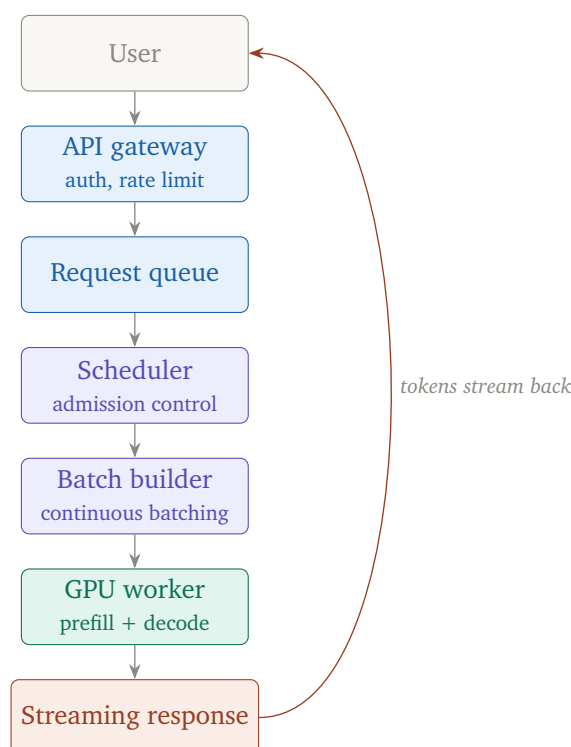
Between your code and the GPU sits a serving engine. Its job is to turn a stream of unpredictable requests into efficient GPU work.



Core responsibilities: queue management, admission control (rejecting or delaying when overloaded), batch formation (continuous batching), and KV-cache memory management (PagedAttention).

13.2 The Request Lifecycle

Here is the full path a single request travels in production, from the user's device to a streaming response.



Two details matter here. First, TTFT is measured from the user’s perspective, so it includes *queue* and *scheduler* delay — not just prefill compute. A request can wait in the queue under load before any GPU work begins. Second, the response streams token by token back to the user as decode proceeds, so the user sees the first word at TTFT and the rest at the tokens/sec rate.

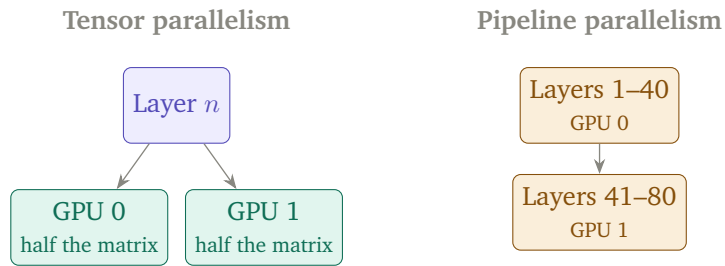
This is why the precise statement is “TTFT is dominated by prefill *once the request reaches the model*.” Under heavy load, queueing delay can dominate TTFT entirely, and no amount of prefill optimization will help — you need more capacity or better admission control.

13.3 The Major Engines

Engine	Distinguishing strengths
vLLM	PagedAttention, continuous batching, broad model support. The default open-source choice.
TGI	HuggingFace’s production server; tight ecosystem integration, token streaming.
SGLang	RadixAttention for aggressive prefix-cache reuse; strong for structured / multi-call workloads.
TensorRT-LLM	NVIDIA’s compiled-kernel engine; highest raw throughput on NVIDIA hardware, more setup cost.

13.4 Distributed Inference

When a model or its KV cache exceeds one GPU, work is split across many.



- **Tensor parallelism:** split each layer's matrices across GPUs; they cooperate on every token. Low latency, heavy interconnect traffic.
- **Pipeline parallelism:** assign contiguous layer ranges to different GPUs; tokens flow through the pipeline. Less interconnect, but pipeline "bubbles" can idle GPUs.

In plain terms Tensor parallelism is several chefs cooking the *same* dish together, each handling part of every step. Pipeline parallelism is an assembly line: chef one does the first half of every dish, then passes it to chef two. Big models use both at once.

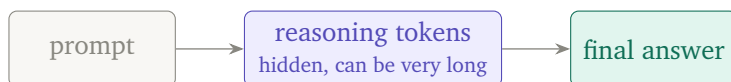
PART VIII

Agents and Reasoning Models

Modern Reasoning Models

14.1 A Different Inference Profile

Standard models map prompt directly to answer. Reasoning models (the o-series, Claude reasoning models, DeepSeek-R1, Qwen reasoning variants) first generate a long internal chain of thought, then a final answer. This dramatically changes inference economics.

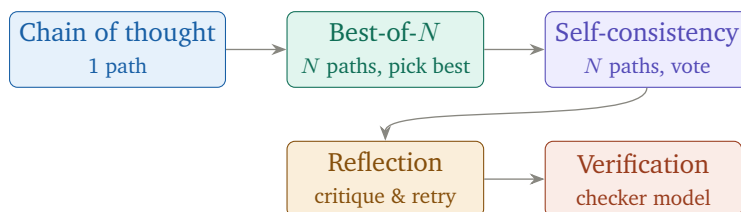


14.2 Core Concepts

- **Chain of thought:** the model writes intermediate reasoning steps before answering. On hard problems (math, logic, multi-step planning) this substantially improves accuracy, because each step conditions the next rather than forcing a single leap to the answer.
- **Test-time compute:** spending more decode tokens at inference to get better answers — trading latency and cost for quality *at run time*, not at training time. This is a genuinely new scaling axis: instead of a bigger model, give the same model more room to think.
- **Reasoning tokens:** the thinking tokens themselves, often hidden from the user but fully billed and fully counted against the context window.

14.3 Strategies for Spending Test-Time Compute

Once you accept that more inference compute can buy more accuracy, the question becomes *how* to spend it. The main strategies form a ladder of increasing cost.



- **Chain of thought:** one reasoning path. Cheapest; already a large gain over direct answering.
- **Best-of- N :** generate N independent answers, then select the best by a scoring function or reward model. Cost scales linearly with N .

- **Self-consistency:** generate N reasoning paths and take a majority vote on the final answer. Exploits the fact that correct reasoning converges while errors scatter. Strong on problems with a single verifiable answer.
- **Reflection:** the model critiques its own draft and revises it. Catches errors a single forward pass misses, at the cost of extra generation rounds.
- **Verification:** a separate checker (a model or a rule-based verifier, e.g. running generated code) validates the answer; failures trigger a retry. Most reliable, most expensive.

Test-time scaling is where much of the recent frontier progress has come from — not from bigger architectures, but from teaching models to use inference compute well. The cost model is the lever every other chapter described: every extra reasoning path, reflection round, or verification pass is more decode tokens, each a full forward pass that grows the KV cache.

Reasoning tokens are still decode tokens — each costs a full forward pass and grows the KV cache. A model that “thinks” for 4,000 tokens before answering pays the full decode cost for all 4,000, and best-of- N or self-consistency multiplies that by N . Test-time compute is real compute, with real latency and a real bill.

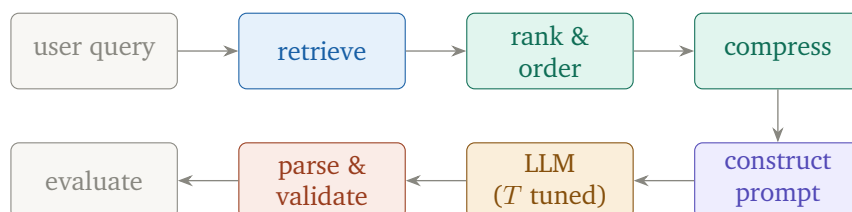
In plain terms A reasoning model is allowed to think out loud on scratch paper before answering. Better answers, but the scratch paper costs real time and money — and it fills the same notebook (context window) the rest of the conversation uses.

PART IX

Synthesis

The Full Stack

15.1 Putting It Together



15.2 Metrics to Track Per Experiment

Category	Metric	Captures
Performance	TTFT	prefill duration; scales with input length
Performance	Tokens/sec	decode throughput; bandwidth-bound
Performance	E2E latency	user-perceived response time
Cost	Input tokens	prefill cost and per-request price
Cost	Output tokens	decode cost and generation time
Quality	Accuracy	correctness on a fixed test set
Quality	Faithfulness	answer grounded in provided context
Quality	Consistency	same query, consistent output

15.3 Benchmarking Summary

Change	TTFT	Tokens/sec	Memory
Longer context	up (prefill $O(N^2)$)	down (decode $O(N)$ bandwidth)	up, linear
Quantization INT8	slight down	$\sim 2\times$ up	halved
Quantization INT4	larger down	$\sim 4\times$ up	quartered
Larger output	no change	no change	grows each step
Batching 1 $\rightarrow$ 8	up	up (GPU util.)	$\times$ batch
Prefix-cache hit	much lower	no change	no change

The “longer context” row has *two different* scalings, not one. **Prefill** latency grows as $O(N^2)$ because of the attention matrix. **Decode** latency grows roughly $O(N)$ because of linear KV-cache bandwidth growth. Do not assume all inference scales quadratically.

Summary: What to Be Able to Explain

Inference

- Why long contexts increase latency: $O(N^2)$ attention in prefill, plus growing KV-cache bandwidth per decode step ($O(N)$).
- How KV caching works: computed once in prefill, read (never recomputed) in decode, grows one row per step.
- Why batching improves throughput: amortizes the fixed cost of loading weights from HBM across many simultaneous decode steps.
- When quantization helps: whenever memory is the binding constraint. INT8 for production, INT4 for edge/CPU.

Sampling

- When to use $T=0$: extraction, classification, structured outputs — anything with a singular correct answer.
- When to use $T \geq 0.8$: brainstorming, creative writing — anything where diversity is valuable.
- Temperature vs. top_p vs. top_k: temperature reshapes the whole distribution; top_p cuts a cumulative-mass window (adaptive); top_k cuts a fixed count (not adaptive).

Context, Evaluation, Systems

- Why context quality often beats model size: the model can only reason over what it receives.
- How retrieval ordering matters: lost-in-the-middle means most-relevant content goes first.
- How to measure quality: offline metrics, LLM-as-Judge, and RAG-specific faithfulness/precision/recall.
- How production serves models: queues, schedulers, continuous batching, PagedAttention, tensor and pipeline parallelism.
- Why reasoning models cost more: reasoning tokens are full decode tokens that consume time, money, and context.

PART X

Hands-On Labs

From Reading to Doing

A concept you have only read stays fragile; a concept you have measured with your own hands stays. These five labs turn the book into a course. Each maps to the parts you have read, states a hypothesis to test, gives runnable starting code, and ends with what you should observe and why.

In plain terms You do not need a cluster. A single consumer GPU (or even a small model on CPU for the sampling and RAG labs) is enough to see every effect in this book first-hand. The goal is not production scale; it is to watch the curves bend the way the theory predicts.

Setup

```
pip install vllm transformers sentence-transformers \
  qdrant-client ragas datasets
# A small model keeps every lab fast:
# Qwen2.5-7B-Instruct or Llama-3.2-3B-Instruct
```


Lab 1 — Measure TTFT

Reinforces: Part I (prefill), Part VII (request lifecycle).

Hypothesis: TTFT scales with input length (prefill cost), while output length barely affects it.

```
import time, requests
def timed(prompt, max_tokens):
    t0 = time.time()
    r = requests.post("http://localhost:8000/v1/completions",
        json={"model": M, "prompt": prompt,
            "max_tokens": max_tokens, "stream": True}, stream=True)
    for line in r.iter_lines():
        if line: return time.time() - t0 # first token arrived
```

Procedure: hold output length fixed at 50 tokens. Run prompts of 100, 500, 2000, 8000 input tokens. Record TTFT for each. Then hold input fixed and vary max_tokens from 50 to 500.

Expected: TTFT rises clearly with input length; TTFT is nearly flat as output length changes.
Why: TTFT is a prefill metric, and prefill cost scales with input tokens, not output tokens.

Lab 2 — Watch KV Cache Growth

Reinforces: Part II (memory bandwidth, KV cache formula).

Hypothesis: VRAM use climbs linearly with sequence length, and decode tokens/sec falls as the cache grows.

```
# In one terminal, sample VRAM every second:
nvidia-smi -query-gpu=memory.used \
  -format=csv -l 1
# In another, generate a long completion and log
# tokens/sec in sliding windows of 100 tokens.
```

Procedure: generate a single 4000-token completion. Plot VRAM over time and tokens/sec over the generation. Compute the per-token KV size from the formula in Part II and compare to the observed slope.

Expected: VRAM rises in a straight line; tokens/sec drifts downward as the sequence lengthens.
Why: each decode step reads a larger KV cache from HBM, and decode is bandwidth-bound.

Lab 3 — Compare Temperatures

Reinforces: Part IV (temperature, top_p, entropy).

Hypothesis: as temperature rises, output diversity rises and determinism falls; top_p only bites at high temperature.

```
from collections import Counter
def diversity(prompt, T, n=20):
    outs = [generate(prompt, temperature=T,
                     top_p=1.0) for _ in range(n)]
    return len(set(outs)) / n # fraction unique
```

Procedure: pick one prompt. For $T \in \{0, 0.2, 0.5, 0.8, 1.2\}$, generate 20 completions each and record the unique fraction. Repeat the $T=1.2$ run with top_p=0.9 and compare.

Expected: unique fraction near 0 at $T=0$ (identical outputs), rising toward 1 at high T . Adding top_p=0.9 at high T reduces incoherent outputs. *Why:* temperature reshapes the distribution; top_p only trims a meaningful tail once the distribution is flat.

Lab 4 — Build a RAG Pipeline

Reinforces: Part V (RAG architecture, embeddings, ordering).

Hypothesis: structure and ordering improve answer quality at equal or lower token cost.

```
from sentence_transformers import SentenceTransformer
from qdrant_client import QdrantClient
emb = SentenceTransformer("all-MiniLM-L6-v2")
# 1. chunk docs 2. embed 3. upsert to Qdrant
# 4. embed query 5. search top-k 6. build prompt
# 7. generate with T=0
```

Procedure: ingest a small document set (say 20 articles). Build three versions from the A/B/C framework: (A) dump all chunks unordered; (B) add Role/Task/Evidence/Format structure; (C) add relevance ordering and compression. Ask the same 10 questions of each.

Expected: B beats A; C beats B, often at fewer tokens. *Why:* structure focuses the model and ordering fights the lost-in-the-middle effect.

Lab 5 — Evaluate the RAG System

Reinforces: Part VI (evaluation, faithfulness, precision/recall).

Hypothesis: the quality you felt in Lab 4 shows up as numbers, and the failure modes localize to specific stages.

```
from ragas import evaluate
from ragas.metrics import (faithfulness,
    answer_relevancy, context_precision,
    context_recall)
result = evaluate(dataset, metrics=[faithfulness,
    answer_relevancy, context_precision,
    context_recall])
```

Procedure: score versions A, B, and C from Lab 4 on all four metrics. Read the metrics diagnostically: low context recall means retrieval missed relevant chunks; low context precision means too much noise was retrieved; low faithfulness means the model hallucinated beyond its context.

Expected: C scores highest overall, and the per-metric breakdown tells you *where* A and B failed. *Why:* each metric isolates one stage of the pipeline, turning a vague “the answer was bad” into an actionable diagnosis.

Where to Go Next

These labs cover the application and serving layers. The natural next step is the systems layer beneath them — a follow-on volume, *LLM Systems Engineering*, covering CUDA fundamentals, GPU architecture, distributed training, tensor and pipeline parallelism internals, MoE routing implementations, vLLM and SGLang internals, speculative-decoding kernels, KV-cache compression, and serving benchmarks. That progression moves you from *understanding* inference systems to *designing and optimizing* them — the level expected of strong research and infrastructure engineers.

Further Reading

- **vLLM:** <https://docs.vllm.ai>
- **HuggingFace TGI:** <https://huggingface.co/docs/text-generation-inference>
- **SGLang:** <https://github.com/sgl-project/sglang>
- **Flash Attention:** <https://github.com/Dao-AILab/flash-attention>
- **TensorRT-LLM:** <https://nvidia.github.io/TensorRT-LLM/>
- **Ragas:** <https://docs.ragas.io>
- **DeepEval:** <https://github.com/confident-ai/deepeval>
- **Anthropic prompt engineering:** <https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/overview>
- **OpenAI prompt engineering:** <https://platform.openai.com/docs/guides/prompt-engineering>
- **LangChain retrieval:** <https://python.langchain.com/docs/concepts/retrieval/>